

Employee Attrition Prediction

1. Introduction:

Organizations face a serious problem with employee attrition, which raises expenses and causes disruption. Proactive measures to keep valuable employees can be made possible by attrition prediction. Using a dataset comprising different employee attributes, this project uses machine learning techniques to create predictive models for employee attrition.

2. Data Description And Exploration:

"IBM-HR-Employee-Attrition.csv" is the dataset used in this analysis. It includes comprehensive information about workers, such as job details,

performance indicators, and demographics. The important key points are following:

First Data Inspection: A pandas DataFrame was loaded with the dataset. To gain a basic understanding of the data structure, data types, and summary statistics, the functions `head()`, `tail()`, `info()`, and `describe()` were utilized.

Missing and Duplicated Values: `isnull().sum()` was used to check the dataset for missing values, and `duplicated().sum()` was used to check for duplicated values. There were no duplicate or missing values discovered.

Data Shape: `df.shape` was used to ascertain the dataset's shape.

Target Variable Distribution: `value_counts()` was used to examine the distribution of the target variable, "Attrition." There was a class imbalance in the dataset, with more employees not attrite.

Exploratory Data Analysis (EDA): Using histograms, visualizations were made to examine the distributions of important numerical variables like age, job satisfaction, years at company, and monthly income. To comprehend the relationships between these features, a heatmap was used to visualize a correlation matrix of numerical features.

3. Data Cleaning And Preparation:

The following actions were taken in order to get the data ready for model training:

Target Variable Encoding: LabelEncoder was used to transform the categorical "Attrition" variable into a numerical representation (0 for "No," 1 for "Yes").

Categorical Feature Encoding: Using `pd.get_dummies`, one-hot encoding was used to identify and transform categorical features into numerical format. Multicollinearity was avoided by using the `drop_first=True` argument.

Outlier Handling: Winsorizing was used to handle outliers in numerical columns, clipping the values at the 1.5 IQR bounds. This procedure did not include the 'Attrition' column.

Data Verification: To ensure that the cleaning procedures were applied successfully, the `info()` and `shape` of the cleaned DataFrame were examined.

4. Model Training, Optimizaton And Evaluation:

Data Splitting: To ensure reproducibility, the prepared data was divided into training and testing sets using `train_test_split` with a 20% test size and a `random_state`.

Feature Scaling: `StandardScaler` was used to scale the numerical features in the training and testing sets. This ensured that each feature had a comparable range, which is crucial for certain machine learning algorithms.

Model Initialization: The models for the Random Forest Classifier (`RandomForestClassifier`) and

Logistic Regression (LogisticRegression) were set up. To guarantee convergence for Logistic Regression, `max_iter` was raised to 2000.

Model Training: The scaled training data was used to train both models.

Hyperparameter Optimization: The best hyperparameters for both models were determined using `GridSearchCV`. Each model had its own parameter grid, and the optimization process was carried out with 5-fold cross-validation and 'roc_auc' as the scoring metric. Each model's optimal hyperparameters were printed.

Model Selection: `best_rf_model` and `best_lr_model` were the names of the top-performing models as determined by the `GridSearchCV` results.

Model Prediction: The top-performing models were used to make predictions on the scaled test set.

Model Evaluation: To compute and print the important evaluation metrics—accuracy, precision, recall, F1-score, and AUC—a function called `evaluate_model` was defined. Both the Random Forest and Logistic Regression models were assessed using the function.

Confusion Matrices: To display the quantity of true positives, true negatives, false positives, and false negatives, confusion matrices for both models were created and displayed using heatmaps.

ROC Curve and AUC: The Area Under the Curve (AUC) was computed and presented after the Receiver Operating Characteristic (ROC) curve was

plotted for both models. The models' capacity to differentiate between the two classes is evaluated with the aid of this visualization.

5. Model Prediction And Results Using LIME:

Model Selection for Explanation: The `best_lr_model` was selected to provide an explanation.

Initialization of the LIME Explainer: The training data, feature names, class names, and the mode "classification" were used to initialize a `LimeTabularExplainer`. For LIME, features were handled as continuous.

Selection of Instance for Explanation: The test set's initial instance was chosen for explanation.

Creating an Explanation: An explanation for the

chosen instance was produced using the explainer's `explain_instance` method, which required the prediction function (`predict_proba`) and the number of features to be shown (`num_features`).

Visualizing the Explanation: A list displaying the various features' contributions to the prediction for the chosen instance was printed from the generated LIME explanation.

6. Visualizations:

Attrition Distribution by Demographics: The distribution of attrition by Age, Marital Status (specifically "Single"), OverTime, and Department (specifically "Research & Development") was visualized using count plots.

Relationship between Attrition and Key Drivers: To

illustrate the connection between Attrition and Years at Company and Monthly Income, box plots were made.

Model Performance Visualization: To give a unified picture of model performance, the confusion matrices and ROC curve with AUC values for both models were plotted once more in the visualization section.

7. Conclusion:

The project effectively created and assessed machine learning models for forecasting employee attrition. Using GridSearchCV to optimize their hyperparameters, the Random Forest and Logistic Regression models both showed respectable performance. The models' advantages and disadvantages were revealed by the evaluation

metrics and visualizations. Individual predictions were successfully explained by LIME, which improved the selected model's interpretability.